附件 2：



华南理工大学

South China University of Technology

课程设计报告书

题目： 二号，黑体，加粗，居中

学　院 _____

专　业 _____

学生姓名 _____

学生学号 _____

指导教师 _____

课程编号 _____

课程学分 _____

起始日期 _____

封面纸推荐用 210g/m2 的绿色色书

教师评语	教师签名： 日期：
成绩评定	
备注	

局域网聊天室软件设计

选题编号 5　　Python 客户端与 C++ 服务端

一 选题背景

局域网多人即时通信（Instant Messaging）是计算机网络中最典型、最综合的应用场景之一，涵盖客户机/服务器（C/S）体系结构、传输层协议特性（TCP 与 UDP）、IP 组播技术、应用层通信协议设计、报文定界与数据完整性校验等核心知识点。本课题面向局域网内部多人文本交互需求，设计并实现了一套具备账号注册、登录认证、注销退出、实时在线列表同步、多人群发以及选定多人精准定向发送（私聊）能力的局域网聊天室系统。

系统采用跨语言协同架构，由 Python 3 图形客户端与 C++17 高性能服务端协同构建。群聊采用高效的 IPv4 组播（Multicast）技术，用户认证与定向私聊采用高可靠的 TCP 单播，并在应用层实现了 4 字节大端长度前缀定界（解决 TCP 粘包拆包问题）、基于 SHA-256 的报文完整性校验、应用层 ACK 确认与超时重传（实现网络抖动环境下的可靠传输保证），以及基于 PBKDF2 的加盐密码安全存储机制。系统既有扎实的网络原理支撑，又具备完整的工程交付成果，包括可运行软件、全套源代码、详细协议设计、12 项自动化网络集成测试及演示录像。

表 1 作业要求与实现映射

要求	实现位置与行为
局域网多人通信	服务端向 239.255.42.99:45861 发送 UDP 组播，TTL=1；带应用层 ACK 与 TCP 补发
客户机与服务器模式	Python Tkinter 图形客户端与 C++17 Winsock2 高性能服务端
中英文文本支持	UTF-8 JSON 编码传输，支持中文账号、中英文正文及多行换行消息
账号与界面交互	图形化注册、登录、注销、在线成员列表动态刷新、聊天记录展示与输入框
群发与选定多人私聊	群发按钮走组播；列表多选好友后走 TCP 定向单播，未选中成员完全隔离
无效任务码与容错处理	应用层完备状态机，遇到无效任务码或错误参数返回统一错误响应且不崩溃
报文完整性防篡改	五字段标准化序列化 + SHA-256 散列校验，单比特变动即拒绝执行业务
规范技术文档与交付物	详细协议设计规范、核心源码附录、自动化测试证据及 720p 演示视频

二 方案论证

2 1 通信方式选型与传输原理

在网络协议选型上，如果群聊全部采用 TCP 单播，当群内有 N 位在线成员时，服务端转发一条群聊消息就必须在 N 条独立的 TCP 连接上分别复制、序列化并发送该数据包 N 次。随着在线人数增加，服务端 CPU 和出口带宽将随人数呈线性增长，容易造成服务器计算压力与局域网网络拥塞。相反，若全部采用 UDP，虽然协议开销低、并发能力强，但 UDP 作为轻量无连接协议，设计目标偏向传输效率而非连接状态维护。在用户注册认证、登录握手与在线成员动态同步等关键业务中，强一致性的时序保证至关重要；同时在定向私聊场景中，TCP 单播能够在传输链路层面天然实现点对点的私密隔离。因此，将面向连接的 TCP 与无连接的 UDP 有机结合，构建双通道互补模型，是兼顾传输性能与业务可靠性的最优方案。

本系统设计的“TCP 控制通道 + UDP 组播群聊”双通道混合传输架构充分发挥了二者的技术优势。IPv4 组播利用 Class D 类特殊地址（本项目使用 239.255.42.99:45861）。服务端转发群发消息时，只需向该地址发送一次数据报，底层交换机与网卡硬件会根据 IGMP 协议自动识别组播成员，在硬件链路层高效复制并分发给所有加入了该组播组的主机，极大地节约了网络带宽资源[1]。而在账号注册、登录认证、在线成员同步以及定向私聊时，则采用面向连接、提供可靠顺序交付的 TCP 协议（端口 45860）。定向私聊通过 TCP 单播仅发送给被选中的客户端 Socket，未被选中的第三方用户在物理网络链路上根本不会收到该数据包，实现了底层网络级别的隐私隔离。

在软件实现分工上，Python 客户端依托内置 Tkinter 快速构建直观友好的图形交互界面，网络通信线程与界面主线程解耦，避免网络阻塞界面；C++17 服务端依托 Windows 原生 Winsock2 API 与多线程，负责高并发网络 I/O 调度、会话认证、组播下发与重传队列管理。两侧遵循统一的规范化 JSON 协议进行报文编解码，直观展示了计算机网络应用层协议解耦不同软硬件平台的本质思想。

2 2 可靠性设计与安全机制原理

为充分发挥 UDP 组播的高吞吐优势，同时保障在偶发网络抖动环境下的消息必达，本设计在应用层构建了轻量级的可靠传输保障机制。服务端向组播组发送群聊消息后，在内存中为每位在线成员记录一条待确认项，并设定 800 毫秒超时时间。客户端收到组播报文并验证无误后，立即通过可靠的 TCP 链路回传确认报文（ACK）。若 800 毫秒内服务端未收到某在线成员的 ACK，判定该成员需要可靠补偿，服务端立即通过

该成员的 TCP 稳定连接将原消息单播补发一次。客户端维护最近接收的消息唯一标识（ID）缓存窗口，若同时收到组播与补发副本，自动过滤重复数据，实现“既享组播高效，又保消息必达”。

针对 TCP 字节流无消息边界的特性，本设计采用“4 字节大端整数长度头 + JSON 数据体”的定界机制。接收端循环读满 4 字节获得后续包体精确字节数 N，再精准读满 N 字节后交付反序列化，彻底解决粘包拆包问题。在数据安全与存储规范上，系统严格遵循现代网络安全准则，采用 16 字节高强度真随机盐值，并使用 PBKDF2-HMAC-SHA256 算法进行 200,000 次高强度迭代生成 32 字节派生散列存储于本地[2]。随机盐消除了彩虹表离线碰撞威胁，高迭代次数大幅拉高了暴力破解的时间成本。报文传输层引入 SHA-256 单向散列校验和，利用其显著的雪崩效应，单比特损坏或修改即导致散列值剧烈改变，接收端能准确识别并丢弃受损或篡改报文，保障数据完整性。

2 3 运行环境与网络配置规范

系统实测环境为 Windows 操作系统、Python 3.10+ 标准环境（测试版本 3.14.7）、MinGW-w64 GCC 16.1.0。客户端仅依赖 Python 标准库，服务端提供预编译的静态可执行文件 bin/chat_server.exe，开箱即用。网络端口默认规划为 TCP 45860（认证与私聊）与 UDP 45861（群聊组播）。

组播数据报的 IP 首部 TTL（Time to Live，生存时间）严格设置为 1。这是计算机网络的核心控制机制：TTL=1 意味着组播数据包每经过一台路由器，TTL 就会减为 0 并被丢弃，从而保证组播通信严格限制在当前本地局域网（同一物理子网）内部，防止广播风暴外溢到广域网。在多电脑联机部署时，所有电脑连接同一 Wi-Fi 或交换机，客户端输入服务端局域网 IPv4 即可联机；若服务端具备多个网卡或虚拟网卡，可通过界面指定局域网网卡 IP，确保组播套接字绑定正确的出口网卡。

三 过程论述

3 1 模块结构与数据流

为清晰说明账号控制通道、定向私聊单播通道与群聊组播通道的数据流向，图 1 给出了系统软件架构与数据流示意图。客户端采用多线程模型，主线程负责界面渲染与用户输入响应，网络后台线程负责套接字轮询接收，并通过线程安全队列将数据事件传递给主线程刷新控件[3]。

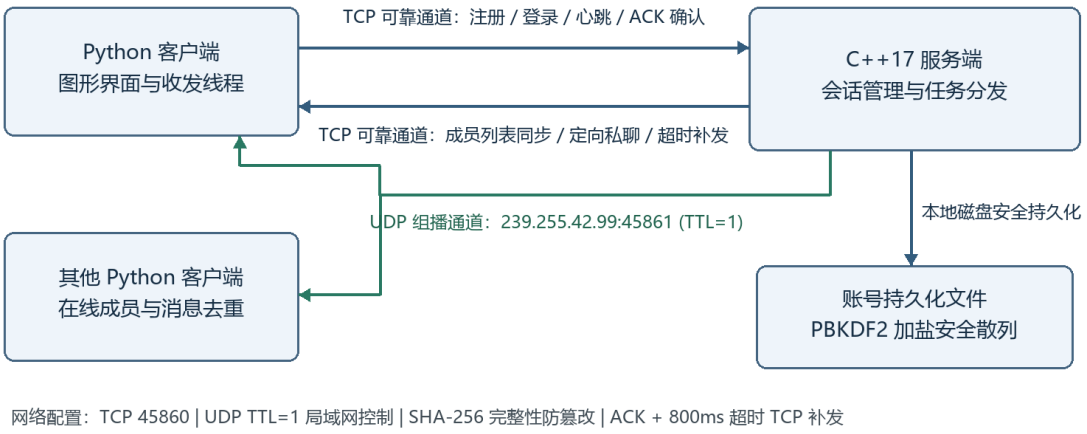


图 1 聊天室模块结构与传输路径全景图

在数据流向中，客户端通过 TCP 链路向服务端发送注册、登录、拉取成员、发送聊天等请求。服务端验证身份后，将群聊数据向 UDP 组播地址广播，将定向私聊直接写入目标成员的 TCP 套接字；用户密码哈希持久化保存在服务端本地磁盘。客户端每 4 秒向服务端发送心跳报文，服务端维护读写超时检测，实现失联主动感知。表 2 汇总了工程核心模块的职责分工。

表 2 工程模块与职责分工

模块路径	实现语言与职责说明
server_cpp/main.cpp	C++17 服务端：底层 Winsock 并发调度、帧解析、会话状态、组播与重传队列
lan_chat/protocol.py	Python 协议层：应用层 LCP 1 编码、定界解码、标准化序列化与 SHA-256 校验
lan_chat/client.py	Python 网络层：TCP Socket 维护、UDP 组播组加入与收发、ACK 回传与消息去重
lan_chat/gui.py	Python 交互层：Tkinter 图形化界面、账号操作、群聊展示、好友多选定向私聊

模块路径	实现语言与职责说明
lan_chat/native_server.py	Python 控制层：管理 C++ 服务端子进程的启动、日志捕获与优雅退出
tests/test_chat.py	自动化测试：12 项真实网络集成测试，检验粘包拆包、组播收发与异常容错

3 2 账号与会话状态流转

服务端采用面向连接的状态机管理客户端生命周期。连接建立后处于未认证状态，仅允许执行注册、登录与心跳请求；注册成功并不自动建立会话，需显式登录。登录成功后，连接与特定用户名绑定并加入全局在线列表，同时防止同一账号在多处重复登录。用户主动注销时，服务端清理会话并移除在线成员，但不删除磁盘账号文件。当网络异常中断或套接字读写超时到达时，服务端安全关闭连接，释放资源并向全网广播成员变动。

在并发控制方面，磁盘账号读写引入互斥锁，并在持久化时写入临时文件再执行原子文件替换，杜绝并发写入损坏文件；每个 TCP 连接的发送操作配置独立写锁，防止多线程同时写入导致数据帧交叉错乱；在线成员列表的推送采用串行化广播，确保新状态不会被过期的旧快照覆盖。

3 3 应用协议语法

应用层协议命名为 LCP 1（LAN Chat Protocol v1）。TCP 帧采用“4 字节大端整数长度 N + N 字节 UTF-8 JSON 文本”的封装格式，N 的有效范围为 1 至 32768 字节。接收函数先循环读满 4 字节，再循环读满正文，彻底消除粘包拆包现象。UDP 组播数据报由于传输层本身保留了报文边界，直接封装 JSON 文本，不含 4 字节长度头。

表 3 报文顶层字段规范

字段	数据类型与规格	设计含义与作用
v	整数，固定为 1	协议版本号，便于未来平滑扩展
id	32 位小写十六进制字符串	独立随机消息唯一标识符，用于去重与 ACK 确认
type	1 至 40 字符大写字符串	业务任务类型码，区分具体操作
ts	0 至 2^53-1 范围的整数	Unix 毫秒时间戳，供界面展示与调试使用
body	JSON 键值对对象	具体业务参数载荷
checksum	64 位小写十六进制字符串	除自身外其余 5 个字段的 SHA-256 完整性散列值

报文顶层规范化定义了表 3 所示的 6 个核心字段，具备严密的格式自洽性。对于格式缺失、字段冗余、非 UTF-8 编码或版本不符的数据，协议栈均予以规范化校验拦

截。JSON 键名按字典序递归排序，采用紧凑分隔符，不转义中文。C++ 解析器设置了 JSON 嵌套深度阈值，防止恶意构造的超深对象消耗系统栈内存。

3 4 客户端任务语义

表 4 客户端请求与状态设计

任务码	body 主要参数	语义说明与前置状态要求
REGISTER	name, password	未登录；用户名 1~20 字符，密码 6~128 字符；加盐哈希写入磁盘
LOGIN	name, password	未登录；验证密码并排重在线；成功返回组播 IP 与端口
LOGOUT	空对象	已登录；注销会话并清理重传项，保留磁盘账号数据
MEMBERS	空对象	已登录；主动请求拉取当前最新在线成员完整列表
CHAT	text, mode=all, targets=[]	已登录；群发消息，由服务端通过 UDP 组播分发给全员
CHAT	text, mode=selected, targets	已登录；定向私聊，仅通过 TCP 单播发送给选中的 1~64 位在线好友
ACK	message_id	已登录；确认收到特定群聊消息，供服务端清除超时重传项
PING	空对象	任意连接状态；发送探活信令，服务端回复 PONG 维持连接

CHAT 消息正文规范为 1 至 2000 个字符且非全空白文本。定向私聊的目标列表会自动去重；若其中包含离线成员，系统实行事务性原子处理并及时提示发送方，保证收发双方认知的一致性。发送者身份由服务端会话上下文统一注入，从根源上保障了发送者身份的真实可信与不可篡改。连接内部维护最近请求 ID 记录，可有效拦截短时间内的重复操作。

3 5 服务端响应与错误处理

表 5 服务端响应类型与字段含义

响应类型	主要字段说明与业务含义
RESULT	request 对应原请求 ID，action 标识操作类型，ok 表示成败，message 为提示文字
登录 RESULT	附加 name、group、udp_port、room_id，用于客户端自动配置并加入组播组
聊天 RESULT	附加 message_id，表示服务端已受理该请求，进入组播下发流程
ROSTER	members 为按字典序排列的完整在线成员名单，客户端收到后全局替换旧名单
MESSAGE	room_id、sender、text、mode、targets、recipients 等，实际接收的消息内容

响应类型	主要字段说明与业务含义
ERROR	code 为机器识别错误码，message 为可读提示，request 为关联的原请求 ID
PONG	心跳响应，维持链路双向保活

表 6 常见错误语义与系统容错策略

错误代码	触发条件	系统应对策略与连接状态
INVALID_TASK	请求的 type 任务码未定义	返回错误响应，保持连接正常，继续处理后续业务
BAD_PACKET	数据非合规 JSON 或 UTF-8 编码错误	丢弃该帧数据，保持连接继续读取
BAD_CHECKSUM	SHA-256 散列值不匹配，数据遭篡改	拒绝执行业务，返回错误，连接正常保留
FRAME_TOO_LARGE	数据帧长度小于等于 0 或大于 32KB	判定为非法异常流量，主动断开异常连接
BAD_BODY	业务参数类型、字符长度不符合规范	拒绝执行当前任务，连接保留
AUTH_FAILED	用户不存在或密码校验错误	认证失败，当前连接保持未认证状态
ALREADY_ONLINE	该账号当前已在其他客户端登录中	拒绝重复登录，避免多连接状态冲突
NOT_LOGGED_IN	未登录状态下尝试执行聊天业务	拦截请求并提示需先完成登录认证
STATE	已登录状态下重复调用登录或注册	提示当前状态冲突，要求先注销
NO_RECIPIENT	定向私聊目标列表为空或包含离线成员	整条消息不予发送并提示检查好友状态
REPLAY	检测到重复提交的请求 ID	拦截重复请求，不重复执行业务
RATE_LIMIT	单连接 1 秒内请求数超过 120 次	触发限流保护，拒绝执行高频请求

3 6 完整性校验结构与算法

发送方在构造报文时，提取 v、id、type、ts、body 五个字段，按键名递归进行字典序排列，以紧凑无多余空格的 UTF-8 编码格式序列化为字节串 M，计算其 SHA-256(M) 散列值，将得到的 32 字节哈希转换为 64 位小写十六进制字符串填入 checksum 字段。接收方在解析报文时，剥离 checksum 字段，执行完全相同的序列化并重新计算散列值进行比对。

报文处理遵循严格的流水线层次：先检查帧长度、再进行 UTF-8 解码与 JSON 解析、验证字段合法性、执行 SHA-256 散列完整性校验、检查任务码合法性、进行会话状态鉴权，最后执行业务逻辑。若单比特修改了 body.text 内容而保留原摘要，服务端能精准捕获 BAD_CHECKSUM；若构造了校验和正确但任务码未定义的报文，返回 INVALID_TASK。两类测试分别验证了语法完整性与语义容错性，服务端均能稳健运行而不退出。

3 7 组播确认与补发流程

当服务端受理群聊请求时，获取当前在线成员快照，为每位在线成员创建以用户名与 MESSAGE.id 为键的待确认条目，截止时间设置为当前系统时间加上 800 毫秒，随后通过 UDP 套接字向组播组发送一次数据报。客户端收到合法消息后，先验证房间 ID 与接收名单，随后立即通过 TCP 连接回传 ACK 确认，并在本地利用滑动窗口去重后将消息渲染到界面。

服务端后台线程每隔 100 毫秒扫描一次待确认表。若超过 800 毫秒仍未收到某成员的 ACK 且该成员仍处于在线状态，服务端立即通过该成员的 TCP 连接将原消息作为单播补发一次，并移除待确认记录。客户端通过消息唯一 ID 进行全局去重，即使因网络抖动先后收到组播与 TCP 补发，也仅在界面渲染一次，在保障高可靠性的同时避免了刷屏干扰。

3 8 图形界面与实际运行

系统图形界面采用清晰紧凑的单窗体布局，将账号认证、连接配置、聊天消息流、在线成员列表与输入发送区合理排布。图 2 展示了多用户登录后的中文群聊效果，图 3 通过多客户端收包记录直观证明了两人定向私聊对旁观者的物理级隔离。界面通过事件驱动模型将网络异步事件传递给主线程刷新控件，交互流畅无卡顿。

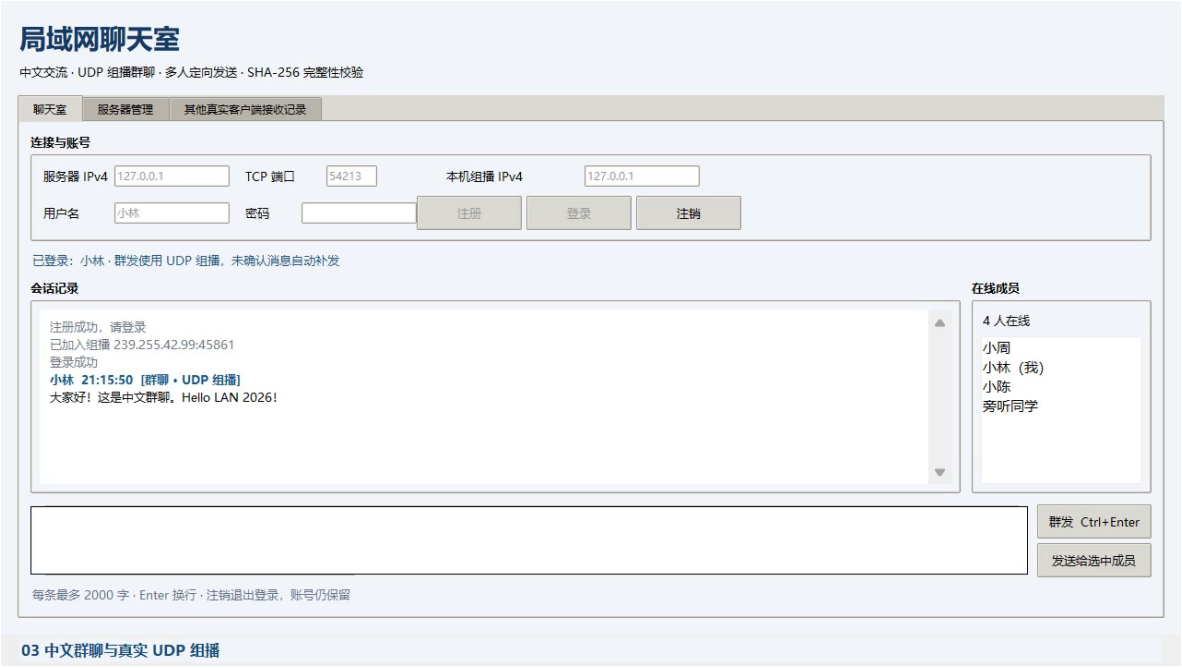


图 2 中文群聊与在线成员界面效果



图 3 两位目标与旁观者的实际接收记录对比

图 2 中的传输标记清晰区分了接收路径（UDP 组播或 TCP 补发），直观证明了局域网组播的实际生效。图 3 中“小周”与“小陈”作为定向私聊的目标成员，实时收到了私密消息；而作为第三方的“旁听同学”界面上仅保留正常的公共群聊，未收到任何私聊报文，验证了定向私聊的精准物理隔离。

四 结果分析

4 1 测试方法与实测结果

系统构建了 12 项覆盖全面的自动化网络集成测试（tests/test_chat.py）。测试程序会真实启动编译好的 C++ 服务端可执行文件，并通过 Python 测试套件建立真实的 TCP 与 UDP 套接字连接进行通信校验。测试使用独立的临时磁盘数据库，执行完成后自动回收进程并清理环境。目前 12 项测试全部通过，测试覆盖了字符集编解码、篡改检测、格式容错、密码加盐、真实组播、私聊隔离、超时补发、粘包拆包定界以及服务端重启持久化等关键网络场景。

表 7 自动化测试覆盖与验证结果

编号	测试检验场景	网络原理与行为	实测结果
01	中英文与多行文本编解码	UTF-8 原生编解码，校验跨语言报文一致性	全部通过
02	报文篡改拦截测试	单比特篡改正文，SHA-256 散列比对失败并安全丢弃	全部通过
03	异常格式与超大报文拦截	格式错误抛弃当前帧，超 32KB 帧断开异常连接	全部通过
04	密码加盐与防重复登录	PBKDF2 随机盐派生散列校验，拦截账号异地重登	全部通过
05	真实 UDP 组播收发与去重	向组播地址发送中文数据报，多客户端接收与窗口去重	全部通过
06	定向多人私聊精准隔离	向 2 位目标发送私聊，第 3 位旁观者无数据包流入	全部通过
07	组播丢包应用层 TCP 补发	客户端退出组播后，800ms 超时触发 TCP 单播补发	全部通过
08	未知任务码与异常注入容错	发送无效 type 后服务端返回错误，连接保持稳定	全部通过
09	TCP 粘包与拆包定界恢复	多帧粘连以及每 7 字节极限拆分，仍 100% 精准解帧	全部通过
10	高危格式与匿名访问拦截	未登录连接拦截聊天业务，连接池正常接纳新连接	全部通过
11	业务边界与离线名单更新	空白正文拦截，好友离线后实时触发名单全网同步	全部通过
12	服务端重启数据持久化验证	服务端进程重启后，读取 accounts.json 成功登录	全部通过

4 2 演示结果与工程解释

在 3 分钟 720p 演示视频中，运行了一个 C++ 服务端与四个客户端实例。公共群聊由全部在线成员同步接收；定向私聊仅在被选中的“小周”与“小陈”的聊天区显示，其余旁观者完全被隔离。当手动退出某客户端的组播组以模拟网络抖动时，后续群聊消息在约 0.8 秒后通过该客户端的 TCP 单播稳定通道自动补发到达，聊天记录标记由 UDP 切换为 TCP，直观展现了应用层可靠性机制的有效性。

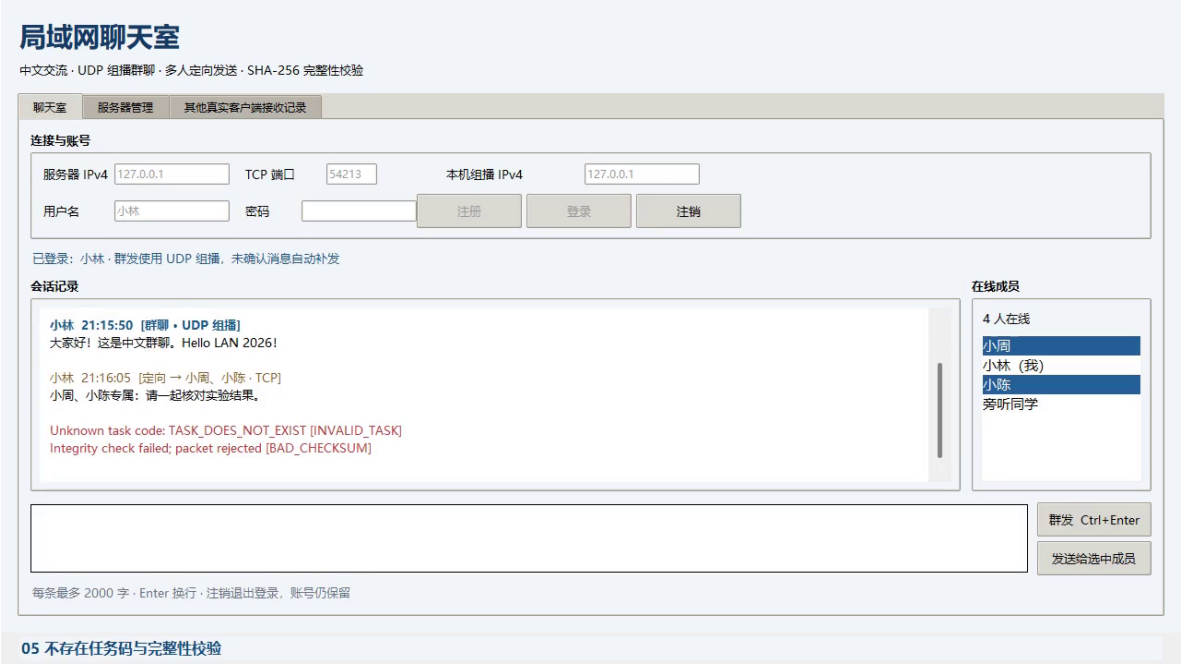


图 4 无效任务码与数据篡改被拒绝后系统继续稳定运行

图 4 展示了系统面对未知任务码与数据完整性校验失败时的容错表现。服务端在拦截非法请求并向客户端返回对应错误代码后，主服务进程与正常客户端连接完全不受影响，继续正常收发心跳与业务消息，具备高度的工程健壮性。

4 3 网络性能分析与部署建议

在网络带宽占用方面，假定群聊在线人数为 N，单条群聊消息长度为 S。若采用传统的纯 TCP 单播群发，服务端需要发出的下行流量为 N * S；而采用 UDP 组播机制后，服务端发出的下行流量始终保持为固定的 1 * S，网络带宽利用率提升了约 N 倍，非常适合多人局域网通信。在实际多机部署时，需确保两台电脑处于同一局域网网段；若校园网 AP 开启了组播隔离，系统内置的 TCP 超时补发机制会自动作为保底手段生效，确保通信通畅。

五 课程设计总结

本课程设计通过结合面向连接的 TCP 控制通道与无连接的 UDP 组播通道，成功设计并实现了一套功能完备、原理清晰、运行健壮的局域网多人聊天室系统。设计贯穿了计算机网络各层核心机制：在传输层深入理解了 TCP 字节流与 UDP 数据报的本质区别，掌握了 IP 组播地址划分及 TTL 本地扩散控制原理；在应用层设计了严密的 4 字节定界机制解决粘包拆包问题，构建了基于 SHA-256 的报文完整性防篡改体系，并通过应用层 ACK 与超时重传机制，实现了弱网环境下的高可靠交付保证。

通过 Python 客户端与 C++ 服务端的跨语言协同开发，加深了对网络协议标准化重要性的认识——只要严格约定报文语法、序列化编码规则与状态机时序，不同语言与平台便能实现无缝互联互通。系统整体设计结构合理、代码规范，未来可在本系统基础上进一步引入 TLS 传输层安全通道、增加公私钥非对称端到端加密、支持断线自动重连与离线消息队列，使局域网通信系统迈向更高的安全性与工业级应用水平。

参考文献

[1] Deering S. Host Extensions for IP Multicasting. RFC 1112, 1989, 第 2 至 3 节. https://www.rfc-editor.org/rfc/rfc1112.txt. 访问日期 2026-10-31.

[2] Moriarty K, Kaliski B, Rusch A. PKCS #5 Password-Based Cryptography Specification Version 2.1. RFC 8018, 2017, 第 5.2 节. https://www.rfc-editor.org/rfc/rfc8018.txt. 访问日期 2026-10-31.

[3] Python Software Foundation. tkinter Python interface to Tcl/Tk, Threading model. https://docs.python.org/3/library/tkinter.html. 访问日期 2026-10-31.

[4] Lohmann N. JSON for Modern C++, version 3.11.3. https://github.com/nlohmann/json/tree/v3.11.3. MIT License. 访问日期 2026-10-31.

附录 A 核心程序完整源码

以下按文件列出本课题核心自编源码，行号对应交付工程。过长行在排版时折行，续行以 > 标识；可执行源码以工程文件为准。第三方 nlohmann/json 3.11.3 头文件体量较大，随 server_cpp/vendor/json.hpp 与 MIT 许可证独立提交，不重复印入本附录。自动测试完整源码另见附录 B。

server_cpp/main.cpp

```
// LCP/1 native Windows server. Build with MinGW-w64 or MSVC, C++17.
#define WIN32_LEAN_AND_MEAN
#define NOMINMAX
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>
#include <bcrypt.h>
#include "vendor/json.hpp"
#include <algorithm>
#include <atomic>
#include <chrono>
#include <deque>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <map>
#include <memory>
#include <mutex>
#include <set>
#include <string>
#include <thread>
#include <vector>

using json = nlohmann::json;
using Clock = std::chrono::steady_clock;
constexpr size_t MAX_FRAME = 32768;
struct Error : std::runtime_error {
    std::string code;
    Error(std::string c, std::string m) : runtime_error(m), code(std::move(c)) {}
};
std::mutex log_mutex;
void log(const std::string& text) {
    std::lock_guard<std::mutex> guard(log_mutex);
    std::cout << text << std::endl;
}
std::string hex(const std::vector<unsigned char>& bytes) {
    const char* digits = "0123456789abcdef";
    std::string result;
    for (auto b : bytes) { result += digits[b >> 4]; result += digits[b & 15]; }
    return result;
}
std::vector<unsigned char> random_bytes(size_t size) {
    std::vector<unsigned char> result(size);
    if (BCryptGenRandom(nullptr, result.data(), ULONG(size), BCRYPT_USE_SYSTEM_PREFERRED_RNG) < 0)
        throw std::runtime_error("BCryptGenRandom failed");
    return result;
}
std::string sha256(const std::string& input) {
    BCRYPT_ALG_HANDLE alg = nullptr;
    BCRYPT_HASH_HANDLE hash = nullptr;
    if (BCryptOpenAlgorithmProvider(&alg, BCRYPT_SHA256_ALGORITHM, nullptr, 0) < 0)
```

```
        throw std::runtime_error("SHA256 provider failed");
    std::vector<unsigned char> output(32);
    NTSTATUS status = BCryptCreateHash(alg, &hash, nullptr, 0, nullptr, 0, 0);
    if (status >= 0) status = BCryptHashData(hash, (PUCHAR)input.data(), ULONG(input.size()), 0);
    if (status >= 0) status = BCryptFinishHash(hash, output.data(), ULONG(output.size()), 0);
    if (hash) BCryptDestroyHash(hash);
    BCryptCloseAlgorithmProvider(alg, 0);
    if (status < 0) throw std::runtime_error("SHA256 failed");
    return hex(output);
}
bool equal_digest(const std::string& a, const std::string& b) {
    if (a.size() != b.size()) return false;
    unsigned char diff = 0;
    for (size_t i = 0; i < a.size(); ++i) diff |= a[i] ^ b[i];
    return diff == 0;
}
std::string derive(const std::string& password, const std::string& salt_hex) {
    if (salt_hex.size() != 32) throw std::runtime_error("invalid account salt");
    std::vector<unsigned char> salt(16), digest(32);
    for (size_t i = 0; i < 16; ++i) salt[i] = (unsigned char)std::stoul(salt_hex.substr(i * 2, 2),
nullptr, 16);
    BCRYPT_ALG_HANDLE alg = nullptr;
    if (BCryptOpenAlgorithmProvider(&alg, BCRYPT_SHA256_ALGORITHM, nullptr, BCRYPT_ALG_HANDLE_HMAC_
FLAG) < 0)
        throw std::runtime_error("PBKDF2 provider failed");
    NTSTATUS result = BCryptDeriveKeyPBKDF2(alg, (PUCHAR)password.data(), ULONG(password.size()),
                          salt.data(), ULONG(salt.size()), 200000, digest.data(), ULONG(digest.siz
e()), 0);
    BCryptCloseAlgorithmProvider(alg, 0);
    if (result < 0) throw std::runtime_error("PBKDF2 failed");
    return hex(digest);
}
// Valid JSON UTF-8 is checked by nlohmann; count Unicode code points.
std::vector<unsigned> codepoints(const std::string& s) {
    std::vector<unsigned> result;
    for (size_t i = 0; i < s.size();) {
        unsigned char c = s[i++]; unsigned cp = c; int count = 0;
        if (c >= 0xf0) { cp = c & 7; count = 3; }
        else if (c >= 0xe0) { cp = c & 15; count = 2; }
        else if (c >= 0xc0) { cp = c & 31; count = 1; }
        while (count-- && i < s.size()) cp = (cp << 6) | (s[i++] & 63);
        result.push_back(cp);
    }
    return result;
}
bool valid_name(const std::string& name) {
    auto chars = codepoints(name);
    if (chars.empty() || chars.size() > 20) return false;
    for (auto c : chars)
        if (!((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9') ||
              c == '_' || c == '-' || (c >= 0x4e00 && c <= 0x9fff))) return false;
    return true;
}
bool hexadecimal(const std::string& s, size_t size) {
    return s.size() == size && s.find_first_not_of("0123456789abcdef") == std::string::npos;
}
json packet(const std::string& type, json body = json::object()) {
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::system_clock::now(
).time_since_epoch()).count();
    json p = {{"v", 1}, {"id", hex(random_bytes(16))}, {"type", type}, {"ts", ms}, {"body", body}};
    p["checksum"] = sha256(p.dump());
    return p;
}
json parse(const std::string& raw) {
    try {
        std::map<int, std::set<std::string>> keys;
```

```
        auto callback = [&keys](int depth, json::parse_event_t event, json& parsed) {
            if (depth > 32) throw Error("BAD_PACKET", "JSON nesting exceeds 32");
            if (event == json::parse_event_t::object_start) keys[depth + 1].clear();
            if (event == json::parse_event_t::key && !keys[depth].insert(parsed.get<std::string>())
 > .second)
                throw Error("BAD_PACKET", "Duplicate JSON key");
            return true;
        };
        json p = json::parse(raw, callback);
        if (!p.is_object() || p.size() != 6 || !p.contains("v") || !p.contains("id") ||
                !p.contains("type") || !p.contains("ts") || !p.contains("body") || !p.contains("che
 > cksum"))
            throw Error("BAD_PACKET", "Invalid envelope fields");
        if (!p["v"].is_number_integer() || p["v"] != 1 || !p["body"].is_object() ||
                !p["id"].is_string() || !hexadecimal(p["id"], 32) || !p["type"].is_string() ||
                p["type"].get<std::string>().empty() || p["type"].get<std::string>().size() > 40 ||
                !p["ts"].is_number_integer() || p["ts"].get<int64_t>() < 0 ||
                p["ts"].get<uint64_t>() > 9007199254740991ULL ||
                !p["checksum"].is_string() || !hexadecimal(p["checksum"], 64))
            throw Error("BAD_PACKET", "Invalid field types or values");
        std::string digest = p["checksum"];
        json unsigned_p = p; unsigned_p.erase("checksum");
        if (!equal_digest(digest, sha256(unsigned_p.dump())))
            throw Error("BAD_CHECKSUM", "Integrity check failed; packet rejected");
        return p;
    } catch (const Error&) { throw; }
    catch (const std::exception&) { throw Error("BAD_PACKET", "Malformed JSON or UTF-8"); }
}
bool read_exact(SOCKET sock, char* data, int size) {
    int offset = 0;
    while (offset < size) {
        int n = recv(sock, data + offset, size - offset, 0);
        if (n <= 0) return false;
        offset += n;
    }
    return true;
}
struct Peer {
    SOCKET sock;
    std::string name;  // Guarded by server state mutex.
    std::mutex write_mutex;
    std::atomic<bool> closed{false};
    std::deque<std::string> seen;
    std::deque<Clock::time_point> recent;
    explicit Peer(SOCKET s) : sock(s) {}
    ~Peer() { closesocket(sock); }
    bool send(const json& p) {
        std::string data = p.dump();
        if (data.size() > MAX_FRAME) throw Error("FRAME_TOO_LARGE", "Response exceeds maximum lengt
 > h");
        uint32_t size = htonl(uint32_t(data.size()));
        std::string frame((char*)&size, 4); frame += data;
        std::lock_guard<std::mutex> guard(write_mutex);
        if (closed) return false;
        size_t offset = 0;
        while (offset < frame.size()) {
            int n = ::send(sock, frame.data() + offset, int(frame.size() - offset), 0);
            if (n <= 0) return false;
            offset += n;
        }
        return true;
    }
};

class Server {
    std::mutex state, account_mutex, roster_mutex;
```

```
177     std::map<std::string, std::shared_ptr<Peer>> users;
178     std::set<std::shared_ptr<Peer>> peers;
179     struct Pending { Clock::time_point deadline; json message; };
180     std::map<std::pair<std::string, std::string>, Pending> pending;
181     std::string room_id = hex(random_bytes(16));
182     std::string db_path;
183     json accounts = json::object();
184     SOCKET listener = INVALID_SOCKET, udp = INVALID_SOCKET;
185     sockaddr_in multicast{};
186 public:
187     explicit Server(std::string path) : db_path(std::move(path)) {
188         if (std::filesystem::exists(std::filesystem::u8path(db_path))) {
189             std::ifstream in(std::filesystem::u8path(db_path));
190             in >> accounts;
191             if (!accounts.is_object()) throw std::runtime_error("Invalid account database");
192         }
193     }
194     void run(const std::string& host, int port, const std::string& iface) {
195         listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
196         sockaddr_in addr{}; addr.sin_family = AF_INET; addr.sin_port = htons((u_short)port);
197         if (inet_pton(AF_INET, host.c_str(), &addr.sin_addr) != 1)
198             throw std::runtime_error("Invalid listening IPv4");
199         if (bind(listener, (sockaddr*)&addr, sizeof(addr)) != 0 || listen(listener, 32) != 0)
200             throw std::runtime_error("Cannot bind/listen TCP port (already in use?)");
201         udp = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
202         DWORD ttl = 1; BOOL loop = TRUE;
203         setsockopt(udp, IPPROTO_IP, IP_MULTICAST_TTL, (char*)&ttl, sizeof(ttl));
204         setsockopt(udp, IPPROTO_IP, IP_MULTICAST_LOOP, (char*)&loop, sizeof(loop));
205         in_addr outgoing{};
206         if (inet_pton(AF_INET, iface.c_str(), &outgoing) != 1)
207             throw std::runtime_error("Invalid multicast interface IPv4");
208         if (setsockopt(udp, IPPROTO_IP, IP_MULTICAST_IF, (char*)&outgoing, sizeof(outgoing)) != 0)
209             throw std::runtime_error("Cannot select multicast interface");
210         multicast.sin_family = AF_INET; multicast.sin_port = htons(45861);
211         inet_pton(AF_INET, "239.255.42.99", &multicast.sin_addr);
212         std::thread([this] { repair(); }).detach();
213         log("READY TCP " + host + ":" + std::to_string(port) + " | UDP 239.255.42.99:45861 | C++17"
  > );
214         for (;;) {
215             SOCKET sock = accept(listener, nullptr, nullptr);
216             if (sock == INVALID_SOCKET) continue;
217             auto peer = std::make_shared<Peer>(sock);
218             DWORD timeout = 15000, send_timeout = 5000;
219             setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, (char*)&timeout, sizeof(timeout));
220             setsockopt(sock, SOL_SOCKET, SO_SNDTIMEO, (char*)&send_timeout, sizeof(send_timeout));
221             {
222                 std::lock_guard<std::mutex> guard(state);
223                 if (peers.size() >= 64) continue;
224                 peers.insert(peer);
225             }
226             std::thread([this, peer] { serve(peer); }).detach();
227         }
228     }
229 private:
230     void error(const std::shared_ptr<Peer>& p, const Error& e, const std::string& rid = "") {
231         p->send(packet("ERROR", {{"code", e.code}, {"message", e.what()}, {"request", rid}}));
232     }
233     void serve(const std::shared_ptr<Peer>& peer) {
234         try {
235             while (!peer->closed) {
236                 uint32_t size;
237                 if (!read_exact(peer->sock, (char*)&size, 4)) break;
238                 size = ntohl(size);
239                 if (!size || size > MAX_FRAME) {
240                     error(peer, Error("FRAME_TOO_LARGE", "TCP frame length must be 1..32768"));
241                     break;
```

```
                }
                std::string raw(size, '\0');
                if (!read_exact(peer->sock, raw.data(), int(size))) break;
                std::string rid;
                try {
                    json p = parse(raw); rid = p["id"];
                    handle(peer, p);
                } catch (const Error& e) {
                    error(peer, e, rid);
                    log("REJECT " + e.code);
                } catch (const json::exception&) {
                    error(peer, Error("BAD_BODY", "Task body has invalid field types"), rid);
                }
            }
        } catch (const std::exception& e) { log(std::string("CONNECTION ERROR ") + e.what()); }
        drop(peer);
    }
    void save_accounts() {
        auto path = std::filesystem::u8path(db_path);
        if (path.has_parent_path()) std::filesystem::create_directories(path.parent_path());
        auto temporary = path; temporary += ".tmp";
        {
            std::ofstream out(temporary, std::ios::binary | std::ios::trunc);
            out << accounts.dump(2); out.flush();
            if (!out) throw std::runtime_error("Cannot write account database");
        }
        if (!MoveFileExW(temporary.c_str(), path.c_str(), MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRIT
 > E_THROUGH))
            throw std::runtime_error("Cannot commit account database");
    }
    void handle(const std::shared_ptr<Peer>& peer, const json& p) {
        std::string task = p["type"], rid = p["id"];
        json body = p["body"];
        static const std::set<std::string> tasks{"REGISTER", "LOGIN", "LOGOUT", "MEMBERS", "CHAT",
 > "ACK", "PING"};
        if (!tasks.count(task)) throw Error("INVALID_TASK", "Unknown task code: " + task);
        if (std::find(peer->seen.begin(), peer->seen.end(), rid) != peer->seen.end())
            throw Error("REPLAY", "Duplicate request identifier");
        peer->seen.push_back(rid); if (peer->seen.size() > 512) peer->seen.pop_front();
        auto now = Clock::now();
        while (!peer->recent.empty() && now - peer->recent.front() > std::chrono::seconds(1)) peer-
 >recent.pop_front();
        if (peer->recent.size() >= 120) throw Error("RATE_LIMIT", "Too many requests");
        peer->recent.push_back(now);
        std::string name;
        { std::lock_guard<std::mutex> guard(state); name = peer->name; }
        if (task == "REGISTER" || task == "LOGIN") {
            if (!name.empty()) throw Error("STATE", "Logout before switching accounts");
            if (!body.contains("name") || !body["name"].is_string() || !body.contains("password") |
 > | !body["password"].is_string())
                throw Error("BAD_BODY", "name and password must be strings");
            std::string requested = body["name"], password = body["password"];
            if (!valid_name(requested) || codepoints(password).size() < 6 || codepoints(password).s
 > ize() > 128)
                throw Error("BAD_BODY", "Name: 1..20 Chinese/ASCII letters/digits/_/-; password: 6.
 > .128 characters");
            if (task == "REGISTER") {
                std::lock_guard<std::mutex> guard(account_mutex);
                bool ok = !accounts.contains(requested);
                if (ok) {
                    auto salt = hex(random_bytes(16));
                    accounts[requested] = {{"salt", salt}, {"digest", derive(password, salt)}};
                    try { save_accounts(); }
                    catch (...) { accounts.erase(requested); throw; }
                }
                peer->send(packet("RESULT", {{"request", rid}, {"action", task}, {"ok", ok},
```

```
                    {"message", ok ? "注册成功，请登录" : "用户名已存在"}}));
                return;
            }
            bool ok = false;
            {
                std::lock_guard<std::mutex> guard(account_mutex);
                if (accounts.contains(requested)) {
                    const json& row = accounts[requested];
                    ok = equal_digest(derive(password, row.at("salt")), row.at("digest"));
                }
            }
            if (!ok) throw Error("AUTH_FAILED", "用户名或密码错误");
            {
                std::lock_guard<std::mutex> guard(state);
                if (users.count(requested)) throw Error("ALREADY_ONLINE", "该账号已经在线");
                peer->name = requested; users[requested] = peer;
            }
            peer->send(packet("RESULT", {{"request", rid}, {"action", task}, {"ok", true}, {"name",
 requested},
                  {"group", "239.255.42.99"}, {"udp_port", 45861}, {"room_id", room_id}, {"message"
, "登录成功"}}));
            log("LOGIN " + requested); roster(); return;
        }
        if (task == "PING") { peer->send(packet("PONG", {{"request", rid}})); return; }
        if (name.empty()) throw Error("NOT_LOGGED_IN", "请先登录");
        if (task == "LOGOUT") {
            {
                std::lock_guard<std::mutex> guard(state);
                users.erase(name); peer->name.clear();
                clear_pending(name);
            }
            peer->send(packet("RESULT", {{"request", rid}, {"action", task}, {"ok", true}, {"messag
e", "已注销登录"}}));
            log("LOGOUT " + name); roster();
        } else if (task == "MEMBERS") { roster(); }
        else if (task == "ACK") {
            if (!body.contains("message_id") || !body["message_id"].is_string()) throw Error("BAD_B
ODY", "message_id must be a string");
            std::lock_guard<std::mutex> guard(state);
            pending.erase({name, body["message_id"].get<std::string>()});
        } else if (task == "CHAT") { chat(peer, body, rid, name); }
    }
    void chat(const std::shared_ptr<Peer>& sender, const json& body, const std::string& rid, const
std::string& name) {
        if (!body.contains("text") || !body["text"].is_string() || !body.contains("mode") || !body[
"mode"].is_string())
            throw Error("BAD_BODY", "text and mode must be strings");
        std::string text = body["text"], mode = body["mode"];
        auto chars = codepoints(text);
        bool blank = chars.empty() || std::all_of(chars.begin(), chars.end(), [](unsigned c) {
            return c == 32 || (c >= 9 && c <= 13) || c == 0xa0 || c == 0x1680 ||
                   (c >= 0x2000 && c <= 0x200a) || c == 0x2028 || c == 0x2029 || c == 0x202f || c =
= 0x205f || c == 0x3000;
        });
        if (blank || chars.size() > 2000) throw Error("BAD_BODY", "消息须为 1 至 2000 个字符且不能
全为空白");
        json targets = body.value("targets", json::array());
        if ((mode != "all" && mode != "selected") || !targets.is_array() || targets.size() > 64)
            throw Error("BAD_BODY", "Invalid mode or targets");
        std::set<std::string> requested;
        for (const auto& target : targets) {
            if (!target.is_string()) throw Error("BAD_BODY", "Recipient must be a string");
            requested.insert(target.get<std::string>());
        }
        std::vector<std::shared_ptr<Peer>> recipients;
        std::vector<std::string> names;
```

```
360         json message;
361         {
362             std::lock_guard<std::mutex> guard(state);
363             if (mode == "all") {
364                 if (!targets.empty()) throw Error("BAD_BODY", "Group message targets must be empty"
 > );
365                 for (const auto& entry : users) names.push_back(entry.first);
366             } else {
367                 if (requested.empty()) throw Error("NO_RECIPIENT", "请选择在线接收人");
368                 for (const auto& target : requested) if (!users.count(target)) throw Error("NO_RECI
 > PIENT", "接收人已离线");
369                 requested.insert(name); names.assign(requested.begin(), requested.end());
370             }
371             for (const auto& target : names) recipients.push_back(users.at(target));
372             message = packet("MESSAGE", {{"room_id", room_id}, {"sender", name}, {"text", text}, {"
 > mode", mode},
373                              {"targets", targets}, {"recipients", names}, {"request", rid}});
374             if (message.dump().size() > MAX_FRAME) throw Error("FRAME_TOO_LARGE", "Encoded message
 > exceeds maximum length");
375             if (mode == "all") for (const auto& target : names)
376                 pending[{target, message["id"].get<std::string>()}] = {Clock::now() + std::chrono::
 > milliseconds(800), message};
377         }
378         if (mode == "all") {
379             std::string data = message.dump();
380             int sent = sendto(udp, data.data(), int(data.size()), 0, (sockaddr*)&multicast, sizeof(
 > multicast));
381             log("MULTICAST " + name + " -> " + std::to_string(names.size()) + " recipients, " +
382                 std::to_string(data.size()) + " bytes" + (sent < 0 ? " (failed; TCP repair pending)
 > " : ""));
383         } else {
384             for (auto& peer : recipients) if (!peer->send(message)) drop(peer);
385             log("SELECTED " + name + " -> " + targets.dump());
386         }
387         sender->send(packet("RESULT", {{"request", rid}, {"action", "CHAT"}, {"ok", true},
388                                 {"message", "服务器已受理"}, {"message_id", message["id"]}}))
 > ;
389     }
390     void clear_pending(const std::string& name) {  // Caller holds state mutex.
391         for (auto it = pending.begin(); it != pending.end();)
392             if (it->first.first == name) it = pending.erase(it); else ++it;
393     }
394     void drop(const std::shared_ptr<Peer>& peer) {
395         if (peer->closed.exchange(true)) return;
396         shutdown(peer->sock, SD_BOTH);
397         bool changed;
398         std::string name;
399         {
400             std::lock_guard<std::mutex> guard(state);
401             name = peer->name; changed = !name.empty();
402             if (changed) { users.erase(name); clear_pending(name); }
403             peer->name.clear(); peers.erase(peer);
404         }
405         if (changed) { log("DISCONNECT " + name); roster(); }
406     }
407     void roster() {
408         std::vector<std::shared_ptr<Peer>> failed;
409         {
410             std::lock_guard<std::mutex> serialize(roster_mutex);
411             std::vector<std::string> names;
412             std::vector<std::shared_ptr<Peer>> clients;
413             {
414                 std::lock_guard<std::mutex> guard(state);
415                 for (const auto& entry : users) { names.push_back(entry.first); clients.push_back(e
 > ntry.second); }
416             }
```

```
            auto message = packet("ROSTER", {{"members", names}});
            for (auto& peer : clients) if (!peer->send(message)) failed.push_back(peer);
        }
        for (auto& peer : failed) drop(peer);
    }
    void repair() {
        for (;;) {
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
            std::vector<std::pair<std::shared_ptr<Peer>, json>> due;
            {
                std::lock_guard<std::mutex> guard(state);
                for (auto it = pending.begin(); it != pending.end();) {
                    if (it->second.deadline <= Clock::now()) {
                        auto user = users.find(it->first.first);
                        if (user != users.end()) due.emplace_back(user->second, it->second.message)
 > ;
                        it = pending.erase(it);
                    } else ++it;
                }
            }
            for (auto& entry : due) {
                if (!entry.first->send(entry.second)) drop(entry.first);
                log("TCP_REPAIR " + entry.second["id"].get<std::string>().substr(0, 8));
            }
        }
    }
};
int wmain(int argc, wchar_t** argv) {
    SetConsoleOutputCP(CP_UTF8);
    WSADATA wsa{};
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) return 1;
    try {
        std::string host = "0.0.0.0", iface = "0.0.0.0", db = "data/accounts.json";
        int port = 45860;
        std::vector<std::string> arguments;
        for (int i = 0; i < argc; ++i) {
            int size = WideCharToMultiByte(CP_UTF8, 0, argv[i], -1, nullptr, 0, nullptr, nullptr);
            std::string value(size, '\0');
            WideCharToMultiByte(CP_UTF8, 0, argv[i], -1, value.data(), size, nullptr, nullptr);
            value.pop_back(); arguments.push_back(value);
        }
        for (int i = 1; i < argc; ++i) {
            std::string arg = arguments[i];
            if (arg == "--help") { std::cout << "server.exe [--host IPv4] [--port N] [--interface I
 > Pv4] [--db PATH]\n"; return 0; }
            if (++i >= argc) throw std::runtime_error("Missing argument value");
            if (arg == "--host") host = arguments[i];
            else if (arg == "--port") port = std::stoi(arguments[i]);
            else if (arg == "--interface") iface = arguments[i];
            else if (arg == "--db") db = arguments[i];
            else throw std::runtime_error("Unknown option: " + arg);
        }
        if (port < 1 || port > 65535) throw std::runtime_error("Port must be 1..65535");
        Server server(db); server.run(host, port, iface);
    } catch (const std::exception& e) { std::cerr << "FATAL " << e.what() << std::endl; return 1; }
    return 0;
}
```

lan_chat/protocol.py

```python
"""LCP/1 framing and integrity checks shared by TCP and UDP."""
import hashlib
import hmac
import json
import math
import re
import struct
import time
import uuid

MAX_FRAME = 32768
MAX_TEXT = 2000
GROUP = "239.255.42.99"
UDP_PORT = 45861
TCP_PORT = 45860


class ProtocolError(ValueError):
    def __init__(self, code, message):
        super().__init__(message)
        self.code = code


def canonical(value):
    return json.dumps(value, ensure_ascii=False, sort_keys=True,
                      separators=(",", ":"), allow_nan=False).encode("utf-8")


def make_packet(task, body=None):
    obj = dict(v=1, id=uuid.uuid4().hex, type=task, ts=int(time.time() * 1000), body=body or {})
    obj["checksum"] = hashlib.sha256(canonical(obj)).hexdigest()
    return obj


def encode(packet):
    data = canonical(packet)
    if len(data) > MAX_FRAME:
        raise ProtocolError("FRAME_TOO_LARGE", "报文超过 32768 字节")
    return data


def _pairs(items):
    result = {}
    for key, value in items:
        if key in result:
            raise ValueError("duplicate key")
        result[key] = value
    return result


def decode(data):
    if len(data) > MAX_FRAME:
        raise ProtocolError("FRAME_TOO_LARGE", "报文过长")
    try:
        obj = json.loads(data.decode("utf-8"), object_pairs_hook=_pairs,
                         parse_constant=lambda x: (_ for _ in ()).throw(ValueError(x)))
        if not isinstance(obj, dict) or set(obj) != {"v", "id", "type", "ts", "body", "checksum"}:
            raise ValueError("fields")
        if type(obj["v"]) is not int or obj["v"] != 1:
            raise ValueError("version")
        if not isinstance(obj["id"], str) or not re.fullmatch(r"[0-9a-f]{32}", obj["id"]):
            raise ValueError("id")
```

```
        if not isinstance(obj["type"], str) or not 1 <= len(obj["type"]) <= 40:
            raise ValueError("type")
        if type(obj["ts"]) is not int or not 0 <= obj["ts"] <= 9007199254740991:
            raise ValueError("timestamp")
        if not isinstance(obj["body"], dict):
            raise ValueError("body")
        checksum = obj["checksum"]
        if not isinstance(checksum, str) or not re.fullmatch(r"[0-9a-f]{64}", checksum):
            raise ValueError("checksum")
        unsigned = {k: v for k, v in obj.items() if k != "checksum"}
        expected = hashlib.sha256(canonical(unsigned)).hexdigest()
        if not hmac.compare_digest(checksum, expected):
            raise ProtocolError("BAD_CHECKSUM", "完整性校验失败，报文已拒绝")
        return obj
    except ProtocolError:
        raise
    except (ValueError, TypeError, UnicodeError, RecursionError, OverflowError) as exc:
        raise ProtocolError("BAD_PACKET", "报文语法或字段格式无效") from exc


def send_frame(sock, packet):
    data = encode(packet)
    sock.sendall(struct.pack("!I", len(data)) + data)


def recv_exact(sock, size):
    data = bytearray()
    while len(data) < size:
        block = sock.recv(size - len(data))
        if not block:
            raise EOFError("连接已关闭")
        data.extend(block)
    return bytes(data)


def recv_frame(sock):
    size = struct.unpack("!I", recv_exact(sock, 4))[0]
    if not 1 <= size <= MAX_FRAME:
        raise ProtocolError("FRAME_TOO_LARGE", "TCP 帧长度不合法")
    return decode(recv_exact(sock, size))
```

lan_chat/client.py

```
"""Headless client used by both Tkinter and integration tests."""
from collections import deque
import socket
import threading

from .protocol import TCP_PORT, ProtocolError, decode, make_packet, recv_frame, send_frame


class ChatClient:
    def __init__(self, callback, interface="0.0.0.0", multicast=True):
        self.callback = callback
        self.interface, self.multicast = interface, multicast
        self.sock = self.udp = None
        self.name = self.room_id = None
        self.lock = threading.Lock()
        self.seen_lock = threading.Lock()
        self.seen, self.order = set(), deque()
        self.stopped = threading.Event()
        self.udp_stopped = threading.Event()

    def connect(self, host, port=TCP_PORT):
        self.sock = socket.create_connection((host, port), timeout=5)
        self.sock.settimeout(None)
        threading.Thread(target=self._receive, daemon=True).start()
        threading.Thread(target=self._heartbeat, daemon=True).start()

    def request(self, task, body=None):
        packet = make_packet(task, body)
        with self.lock:
            if not self.sock or self.stopped.is_set():
                raise OSError("尚未连接服务器")
            send_frame(self.sock, packet)
        return packet["id"]

    def register(self, name, password):
        return self.request("REGISTER", dict(name=name, password=password))

    def login(self, name, password):
        return self.request("LOGIN", dict(name=name, password=password))

    def chat(self, text, targets=None):
        return self.request("CHAT", dict(text=text, mode="selected" if targets is not None else "al
 > l",
                                         targets=targets or []))

    def logout(self):
        return self.request("LOGOUT")

    def _heartbeat(self):
        while not self.stopped.wait(4):
            try:
                self.request("PING")
            except OSError:
                self.close()
                return

    def _receive(self):
        try:
            while not self.stopped.is_set():
                try:
                    packet = recv_frame(self.sock)
                except ProtocolError as exc:
```

```
                    self.callback("warning", dict(message=str(exc)))
                    if exc.code == "FRAME_TOO_LARGE":
                        break
                    continue
                body = packet["body"]
                if packet["type"] == "RESULT" and body.get("ok"):
                    if body.get("action") == "LOGIN":
                        self.name, self.room_id = body["name"], body["room_id"]
                        self._join(body["group"], body["udp_port"])
                    elif body.get("action") == "LOGOUT":
                        self.name = self.room_id = None
                        self._leave()
                if packet["type"] == "MESSAGE":
                    self._message(packet, "TCP")
                elif packet["type"] != "PONG":
                    self.callback(packet["type"].lower(), body)
        except (OSError, EOFError) as exc:
            if not self.stopped.is_set():
                self.callback("warning", dict(message=f"连接中断：{exc}"))
        finally:
            self.close()

    def _join(self, group, port):
        self._leave()
        if not self.multicast:
            self.callback("transport", dict(message="组播已关闭，使用 TCP 补发"))
            return
        udp = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
        try:
            udp.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            udp.bind(("", port))
            udp.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP,
                           socket.inet_aton(group) + socket.inet_aton(self.interface))
            udp.settimeout(0.5)
        except OSError as exc:
            udp.close()
            self.callback("warning", dict(message=f"加入组播失败，自动使用 TCP 补发：{exc}"))
            return
        self.udp = udp
        self.udp_stopped = threading.Event()
        threading.Thread(target=self._receive_udp, args=(udp, self.udp_stopped), daemon=True).start
 > ()
        self.callback("transport", dict(message=f"已加入组播 {group}:{port}"))

    def _leave(self):
        self.udp_stopped.set()
        if self.udp:
            self.udp.close()
            self.udp = None

    def _receive_udp(self, udp, stopped):
        while not stopped.is_set() and not self.stopped.is_set():
            try:
                data, _ = udp.recvfrom(32769)
                packet = decode(data)
                if packet["type"] == "MESSAGE":
                    self._message(packet, "UDP 组播")
            except socket.timeout:
                continue
            except ProtocolError as exc:
                self.callback("warning", dict(message=f"丢弃异常组播：{exc}"))
            except OSError:
                break

    def _message(self, packet, transport):
        body, mid = packet["body"], packet["id"]
```

```
        if (body.get("room_id") != self.room_id or self.name not in body.get("recipients", [])
                or not isinstance(body.get("text"), str)):
            return
        if body.get("mode") == "all":
            try:
                self.request("ACK", dict(message_id=mid))
            except OSError:
                return
        with self.seen_lock:
            if mid in self.seen:
                return
            self.seen.add(mid)
            self.order.append(mid)
            if len(self.order) > 2048:
                self.seen.discard(self.order.popleft())
        self.callback("message", dict(body, transport=transport, id=mid, ts=packet["ts"]))

    def close(self):
        with self.lock:
            if self.stopped.is_set():
                return
            self.stopped.set()
            self._leave()
            if self.sock:
                try:
                    self.sock.shutdown(socket.SHUT_RDWR)
                except OSError:
                    pass
                self.sock.close()
        self.name = self.room_id = None
        self.callback("disconnected", {})
```

lan_chat/gui.py

```
"""Chinese Tkinter GUI. Network workers never access Tk widgets."""
import argparse
from datetime import datetime
from pathlib import Path
import queue
import threading
import tkinter as tk
from tkinter import messagebox, scrolledtext, ttk

from .client import ChatClient
from .protocol import MAX_TEXT, TCP_PORT, ProtocolError
from .native_server import NativeServer


class ChatApp:
    def __init__(self, root, data="data", title="局域网聊天室"):
        self.root, self.data = root, Path(data)
        self.root.title(title)
        self.root.geometry("1080x750")
        self.root.minsize(850, 650)
        self.events = queue.Queue()
        self.client = self.server = None
        self.busy = False
        self.generation = 0
        self.members = []
        self.host = tk.StringVar(value="127.0.0.1")
        self.port = tk.StringVar(value=str(TCP_PORT))
        self.interface = tk.StringVar(value="0.0.0.0")
        self.username = tk.StringVar()
        self.password = tk.StringVar()
        self.status = tk.StringVar(value="未登录 · 在服务器电脑启动服务后连接")
        self.server_host = tk.StringVar(value="0.0.0.0")
        self.server_port = tk.StringVar(value=str(TCP_PORT))
        self.server_interface = tk.StringVar(value="0.0.0.0")
        style = ttk.Style()
        style.theme_use("clam")
        style.configure("TFrame", background="#f3f6fa")
        style.configure("TLabel", background="#f3f6fa", font=("Microsoft YaHei UI", 10))
        style.configure("TButton", font=("Microsoft YaHei UI", 10), padding=(10, 6))
        style.configure("Header.TLabel", font=("Microsoft YaHei UI", 22, "bold"), foreground="#173c
> 66")
        style.configure("TLabelframe", background="#f3f6fa")
        style.configure("TLabelframe.Label", background="#f3f6fa", font=("Microsoft YaHei UI", 10,
> "bold"))
        self._build()
        root.protocol("WM_DELETE_WINDOW", self.close)
        root.after(60, self._poll)

    def _build(self):
        outer = ttk.Frame(self.root, padding=18)
        outer.pack(fill="both", expand=True)
        ttk.Label(outer, text="局域网聊天室", style="Header.TLabel").pack(anchor="w")
        ttk.Label(outer, text="中文交流 · UDP 组播群聊 · 多人定向发送 · SHA-256 完整性校验").pack(a
> nchor="w", pady=(4, 14))
        tabs = ttk.Notebook(outer)
        tabs.pack(fill="both", expand=True)
        chat, monitor = ttk.Frame(tabs, padding=12), ttk.Frame(tabs, padding=12)
        tabs.add(chat, text="  聊天室  ")
        tabs.add(monitor, text="  服务器管理  ")
        self.tabs = tabs
        box = ttk.LabelFrame(chat, text="连接与账号", padding=10)
        box.pack(fill="x")
```

```
        fields = [("服务器 IPv4", self.host, 17), ("TCP 端口", self.port, 7),
                  ("本机组播 IPv4", self.interface, 17)]
        self.connection_entries = []
        for col, (label, var, width) in enumerate(fields):
            ttk.Label(box, text=label).grid(row=0, column=2*col, padx=(0, 5))
            entry = ttk.Entry(box, textvariable=var, width=width)
            entry.grid(row=0, column=2*col+1, padx=(0, 15))
            self.connection_entries.append(entry)
        ttk.Label(box, text="用户名").grid(row=1, column=0, pady=(10, 0), sticky="w")
        self.name_entry = ttk.Entry(box, textvariable=self.username, width=17)
        self.name_entry.grid(row=1, column=1, pady=(10, 0), sticky="w")
        ttk.Label(box, text="密码").grid(row=1, column=2, pady=(10, 0), sticky="w")
        self.pass_entry = ttk.Entry(box, textvariable=self.password, show="●", width=17)
        self.pass_entry.grid(row=1, column=3, pady=(10, 0), sticky="w")
        actions = ttk.Frame(box)
        actions.grid(row=1, column=4, columnspan=2, pady=(10, 0), sticky="w")
        self.register_button = ttk.Button(actions, text="注册", command=lambda: self.auth("REGISTER
"))
        self.register_button.pack(side="left", padx=(0, 5))
        self.login_button = ttk.Button(actions, text="登录", command=lambda: self.auth("LOGIN"))
        self.login_button.pack(side="left", padx=(0, 5))
        self.logout_button = ttk.Button(actions, text="注销", command=self.logout, state="disabled"
)
        self.logout_button.pack(side="left")
        ttk.Label(chat, textvariable=self.status, foreground="#245780").pack(fill="x", pady=9)
        body = ttk.Frame(chat)
        history_box = ttk.LabelFrame(body, text="会话记录", padding=8)
        history_box.pack(side="left", fill="both", expand=True, padx=(0, 12))
        self.history = scrolledtext.ScrolledText(history_box, height=12, width=60, wrap="word", sta
te="disabled",
                                 font=("Microsoft YaHei UI", 10), bg="white", relief="flat", padx=10,
pady=8)
        self.history.pack(fill="both", expand=True)
        self.history.tag_configure("system", foreground="#6d7784")
        self.history.tag_configure("sender", foreground="#175c8c", font=("Microsoft YaHei UI", 10,
"bold"))
        self.history.tag_configure("private", foreground="#84602a")
        self.history.tag_configure("error", foreground="#b13e40")
        sidebar = ttk.LabelFrame(body, text="在线成员", padding=8)
        sidebar.pack(side="right", fill="y")
        self.member_label = ttk.Label(sidebar, text="0 人在线")
        self.member_label.pack(anchor="w", pady=(0, 6))
        self.member_list = tk.Listbox(sidebar, selectmode="multiple", exportselection=False,
                                      width=19, font=("Microsoft YaHei UI", 11), relief="flat",
                                      selectbackground="#265f96", activestyle="none")
        self.member_list.pack(fill="both", expand=True)
        ttk.Label(sidebar, text="单击可多选\n 选中成员后定向发送", foreground="#677485").pack(anchor
="w", pady=(8, 0))
        composer = ttk.Frame(chat)
        self.composer = tk.Text(composer, height=3, wrap="word", font=("Microsoft YaHei UI", 11),
                                relief="solid", borderwidth=1, padx=8, pady=6)
        self.composer.pack(side="left", fill="x", expand=True, padx=(0, 10))
        self.composer.bind("<Control-Return>", lambda event: self.send(False))
        buttons = ttk.Frame(composer)
        buttons.pack(side="right", fill="y")
        self.all_button = ttk.Button(buttons, text="群发  Ctrl+Enter", command=lambda: self.send(Fa
lse), state="disabled")
        self.all_button.pack(fill="x", pady=(0, 5))
        self.selected_button = ttk.Button(buttons, text="发送给选中成员", command=lambda: self.send
(True), state="disabled")
        self.selected_button.pack(fill="x")
        ttk.Label(chat, text="每条最多 2000 字 · Enter 换行 · 注销退出登录，账号仍保留", foreground
="#677485").pack(side="bottom", anchor="w", pady=(7, 0))
        composer.pack(side="bottom", fill="x", pady=(12, 0))
        body.pack(fill="both", expand=True)
        config = ttk.LabelFrame(monitor, text="服务配置", padding=12)
```

```
        config.pack(fill="x")
        for row, (label, var) in enumerate([("监听 IPv4", self.server_host), ("TCP 端口", self.serv
> er_port),
                                            ("组播出口 IPv4", self.server_interface)]):
            ttk.Label(config, text=label).grid(row=row, column=0, sticky="w", pady=5)
            ttk.Entry(config, textvariable=var, width=24).grid(row=row, column=1, padx=10, pady=5)
        self.start_button = ttk.Button(config, text="启动服务器", command=self.start_server)
        self.start_button.grid(row=0, column=2, padx=14)
        self.stop_button = ttk.Button(config, text="停止服务器", command=self.stop_server, state="d
> isabled")
        self.stop_button.grid(row=1, column=2, padx=14)
        ttk.Label(monitor, text="同一局域网运行一个服务器。其他电脑填写服务器的局域网 IPv4；多网卡
> 时填写实际网卡地址。",
                  wraplength=840).pack(anchor="w", pady=12)
        self.server_log = scrolledtext.ScrolledText(monitor, state="disabled", wrap="word",
                    font=("Consolas", 11), bg="#16283d", fg="#d9e9f8", padx=12, pady=12)
        self.server_log.pack(fill="both", expand=True)

    def append(self, text, tag="system"):
        self.history.configure(state="normal")
        self.history.insert("end", text + "\n", tag)
        self.history.see("end")
        self.history.configure(state="disabled")

    def _online(self, online):
        for widget in (self.name_entry, self.pass_entry, self.register_button, self.login_button, *
> self.connection_entries):
            widget.configure(state="disabled" if online else "normal")
        for widget in (self.logout_button, self.all_button, self.selected_button):
            widget.configure(state="normal" if online else "disabled")

    def _validate_connection(self):
        port = int(self.port.get())
        if not 1 <= port <= 65535:
            raise ValueError("端口须在 1 至 65535 之间")
        return self.host.get().strip(), port, self.interface.get().strip()

    def auth(self, action):
        if self.busy:
            return
        name, password = self.username.get().strip(), self.password.get()
        if not name or not password:
            self.append("请填写用户名和密码", "error")
            return
        try:
            host, port, interface = self._validate_connection()
        except ValueError as exc:
            self.append(str(exc), "error")
            return
        # One connection per auth attempt avoids stale server/interface settings.
        if self.client:
            self.generation += 1
            self.client.close()
        self.generation += 1
        generation = self.generation
        self.busy = True
        self.status.set("正在连接服务器…")
        self.client = ChatClient(lambda event, body: self.events.put((generation, event, body)), in
> terface)
        client = self.client

        def work():
            try:
                client.connect(host, port)
                client.request(action, dict(name=name, password=password))
            except (OSError, ValueError) as exc:
```

```
                self.events.put((generation, "warning", dict(message=str(exc))))
                client.close()
        threading.Thread(target=work, daemon=True).start()

    def logout(self):
        try:
            self.client.logout()
        except (OSError, AttributeError) as exc:
            self.append(str(exc), "error")

    def send(self, selected):
        text = self.composer.get("1.0", "end-1c")
        targets = [self.members[i] for i in self.member_list.curselection()] if selected else None
        if selected and not targets:
            self.append("请先在成员列表中选择一个或多个接收人", "error")
            return "break"
        if not text.strip() or len(text) > MAX_TEXT:
            self.append("消息须为 1 至 2000 个字符且不能全为空白", "error")
            return "break"
        try:
            self.client.chat(text, targets)
            self.composer.delete("1.0", "end")
        except (OSError, AttributeError, ProtocolError) as exc:
            self.append(f"发送失败：{exc}", "error")
        return "break"

    def start_server(self):
        if self.server:
            return
        try:
            port = int(self.server_port.get())
            if not 1 <= port <= 65535:
                raise ValueError("端口须在 1 至 65535 之间")
            self.server = NativeServer(host=self.server_host.get().strip(), port=port,
                    interface=self.server_interface.get().strip(), db=self.data / "accounts.json",
                    log=lambda text: self.events.put((None, "server_log", dict(message=text))),
                    on_exit=lambda: self.events.put((None, "server_stopped", {})))
            self.server.start()
            self.start_button.configure(state="disabled")
            self.stop_button.configure(state="normal")
        except (OSError, ValueError) as exc:
            self.server = None
            messagebox.showerror("服务器启动失败", str(exc), parent=self.root)

    def stop_server(self):
        server = self.server
        if server:
            threading.Thread(target=server.stop).start()
        self.stop_button.configure(state="disabled")

    def _poll(self):
        for _ in range(150):
            try:
                generation, event, body = self.events.get_nowait()
            except queue.Empty:
                break
            if generation is not None and generation != self.generation:
                continue
            if event == "server_stopped":
                if self.server and self.server.process.poll() is not None:
                    self.server = None
                    self.start_button.configure(state="normal")
                    self.stop_button.configure(state="disabled")
            elif event == "server_log":
                self.server_log.configure(state="normal")
                self.server_log.insert("end", datetime.now().strftime("%H:%M:%S  ") + body["message
```

```
> "] + "\n")
244                 self.server_log.see("end")
245                 self.server_log.configure(state="disabled")
246             elif event == "result":
247                 self.busy = False
248                 action = body.get("action")
249                 if action == "CHAT":
250                     continue
251                 self.append(body.get("message", ""))
252                 if body.get("ok") and action == "LOGIN":
253                     self._online(True)
254                     self.password.set("")
255                     self.status.set(f"已登录：{body['name']} · 群发使用 UDP 组播，未确认消息自动补
> 发")
256                 elif action == "LOGOUT":
257                     self._online(False)
258                     self.member_list.delete(0, "end")
259                     self.members = []
260                     self.member_label.configure(text="0 人在线")
261                     self.status.set("已注销 · 可以重新登录或切换账号")
262                 elif action == "REGISTER":
263                     self.status.set("注册成功，请点击登录" if body.get("ok") else "注册未完成，请检
> 查提示")
264             elif event == "roster":
265                 chosen = {self.members[i] for i in self.member_list.curselection()}
266                 self.members = body["members"]
267                 self.member_list.delete(0, "end")
268                 for i, name in enumerate(self.members):
269                     self.member_list.insert("end", name + ("（我）" if self.client and name == self
> .client.name else ""))
270                     if name in chosen:
271                         self.member_list.selection_set(i)
272                 self.member_label.configure(text=f"{len(self.members)} 人在线")
273             elif event == "message":
274                 when = datetime.fromtimestamp(body["ts"] / 1000).strftime("%H:%M:%S")
275                 scope = "群聊" if body["mode"] == "all" else "定向 → " + "、".join(body["targets"])
276                 self.append(f"{body['sender']}  {when}  [{scope} · {body['transport']}]", "sender"
> if body["mode"] == "all" else "private")
277                 self.append(body["text"] + "\n", "content")
278             elif event in {"error", "warning"}:
279                 self.busy = False
280                 self.append(body.get("message", "未知错误") + (f" [{body['code']}]" if "code" in bo
> dy else ""), "error")
281                 if not self.client or not self.client.name:
282                     self.status.set("操作未完成，请检查提示后重试")
283             elif event == "transport":
284                 self.append(body["message"])
285             elif event == "disconnected":
286                 self.busy = False
287                 self._online(False)
288                 self.members = []
289                 self.member_list.delete(0, "end")
290                 self.member_label.configure(text="0 人在线")
291                 self.status.set("连接已关闭 · 可以重新登录")
292         self.root.after(60, self._poll)
293 
294     def close(self):
295         if self.client:
296             self.client.close()
297         if self.server:
298             self.server.stop()
299         self.root.destroy()
300 
301 
302 def main():
303     parser = argparse.ArgumentParser(description="局域网聊天室图形界面")
```

```
    parser.add_argument("--data", default="data")
    args = parser.parse_args()
    root = tk.Tk()
    ChatApp(root, args.data)
    root.mainloop()


if __name__ == "__main__":
    main()
```

lan_chat/native_server.py

```
"""Launch and monitor the bundled C++ server without a console window."""
from pathlib import Path
import subprocess
import threading


class NativeServer:
    def __init__(self, host, port, db, interface, log=print, on_exit=lambda: None):
        self.host, self.port, self.db, self.interface = host, port, db, interface
        self.log, self.on_exit = log, on_exit
        self.process = None
        self.monitor = None

    def start(self):
        executable = Path(__file__).resolve().parent.parent / "bin" / "chat_server.exe"
        if not executable.exists():
            raise OSError("找不到 C++ 服务端，请先运行 build_server.bat")
        self.process = subprocess.Popen(
            [str(executable), "--host", self.host, "--port", str(self.port),
             "--interface", self.interface, "--db", str(self.db)],
            stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
            text=True, encoding="utf-8", errors="replace",
            creationflags=getattr(subprocess, "CREATE_NO_WINDOW", 0))
        ready = threading.Event()
        failure = []
        process = self.process

        def monitor():
            for line in process.stdout:
                line = line.rstrip()
                self.log(line)
                if line.startswith("READY "):
                    ready.set()
                if line.startswith("FATAL "):
                    failure.append(line)
            process.wait()
            process.stdout.close()
            ready.set()
            self.on_exit()
        self.monitor = threading.Thread(target=monitor, daemon=True)
        self.monitor.start()
        if not ready.wait(5) or failure or self.process.poll() is not None:
            self.stop()
            raise OSError(failure[0] if failure else "C++ 服务端未能启动，请检查端口和网卡地址")

    def stop(self):
        if self.process and self.process.poll() is None:
            self.process.terminate()
            try:
                self.process.wait(timeout=3)
            except subprocess.TimeoutExpired:
                self.process.kill()
                self.process.wait(timeout=3)
        if self.monitor and self.monitor is not threading.current_thread():
            self.monitor.join(timeout=2)

```

启动与构建文件

lan_chat/__init__.py

```
"""LAN classroom chat. Standard library only."""

__version__ = "1.0.0"
```

lan_chat/__main__.py

```
from .gui import main

main()
```

start.bat

```
@echo off
chcp 65001 >nul
cd /d "%~dp0"
python -m lan_chat
if errorlevel 1 pause
```

start_server.bat

```
@echo off
chcp 65001 >nul
cd /d "%~dp0"
bin\chat_server.exe %*
if errorlevel 1 pause
```

build_server.bat

```
@echo off
chcp 65001 >nul
cd /d "%~dp0"
if not exist bin mkdir bin
where g++ >nul 2>nul
if errorlevel 1 (
  echo 请安装 MinGW-w64 并将 g++ 添加到 PATH，或使用 Visual Studio 开发者命令行：
  echo cl /std:c++17 /EHsc /utf-8 server_cpp\main.cpp /Fe:bin\chat_server.exe ws2_32.lib bcrypt.lib
  exit /b 1
)
g++ -std=c++17 -municode -O2 -Wall -Wextra -finput-charset=UTF-8 -fexec-charset=UTF-8 -static -stat
> ic-libgcc -static-libstdc++ server_cpp\main.cpp -o bin\chat_server.exe -lws2_32 -lbcrypt
if errorlevel 1 exit /b 1
echo C++ 服务端编译完成：bin\chat_server.exe
```

附录 B 自动测试完整源码

tests/test_chat.py

```python
"""Real C++ process + TCP/UDP integration tests, no simulated server."""
from collections import deque
import hashlib
import json
from pathlib import Path
import socket
import struct
import tempfile
import threading
import time
import unittest

from lan_chat.client import ChatClient
from lan_chat.native_server import NativeServer
from lan_chat.protocol import (MAX_FRAME, ProtocolError, canonical, decode, encode,
                               make_packet, recv_frame, send_frame)


class Inbox:
    def __init__(self):
        self.events = deque()
        self.condition = threading.Condition()

    def put(self, event, body):
        with self.condition:
            self.events.append((event, body))
            self.condition.notify_all()

    def wait(self, event, check=lambda body: True, timeout=5):
        deadline = time.monotonic() + timeout
        with self.condition:
            while True:
                for item in list(self.events):
                    if item[0] == event and check(item[1]):
                        self.events.remove(item)
                        return item[1]
                remaining = deadline - time.monotonic()
                if remaining <= 0:
                    raise AssertionError(f"Timed out waiting for {event}: {list(self.events)}")
                self.condition.wait(remaining)

    def has(self, event, check=lambda body: True):
        with self.condition:
            return any(e == event and check(b) for e, b in self.events)


class ProtocolTests(unittest.TestCase):
    def test_01_chinese_utf8_roundtrip(self):
        packet = make_packet("CHAT", dict(text="你好，局域网！Hello 🌏\n第二行"))
        self.assertEqual(decode(encode(packet)), packet)

    def test_02_tampering_detected(self):
        packet = make_packet("CHAT", dict(text="原文"))
        packet["body"]["text"] = "篡改"
        with self.assertRaises(ProtocolError) as ctx:
            decode(canonical(packet))
        self.assertEqual(ctx.exception.code, "BAD_CHECKSUM")

    def test_03_malformed_envelopes(self):
```

```
        for raw in (b"[]", b"{", b"\xff", b'{"v":1,"v":1}', b'{"x":NaN}'):
            with self.subTest(raw=raw), self.assertRaises(ProtocolError):
                decode(raw)
        with self.assertRaises(ProtocolError):
            decode(b" " * (MAX_FRAME + 1))


class NativeIntegrationTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.temp = tempfile.TemporaryDirectory(prefix="lan_chat_test_")
        cls.db = Path(cls.temp.name) / "中文账户" / "accounts.json"
        with socket.socket() as probe:
            probe.bind(("127.0.0.1", 0))
            cls.port = probe.getsockname()[1]
        cls.logs = []
        cls.server = NativeServer("127.0.0.1", cls.port, cls.db, "127.0.0.1", cls.logs.append)
        cls.server.start()

    @classmethod
    def tearDownClass(cls):
        cls.server.stop()
        cls.temp.cleanup()

    def setUp(self):
        self.clients = []
        self.raw = []

    def tearDown(self):
        for client in self.clients:
            client.close()
        for sock in self.raw:
            sock.close()

    def account(self, name, multicast=True):
        inbox = Inbox()
        client = ChatClient(inbox.put, interface="127.0.0.1", multicast=multicast)
        self.clients.append(client)
        client.connect("127.0.0.1", self.port)
        client.register(name, "demo123456")
        result = inbox.wait("result", lambda b: b["action"] == "REGISTER")
        self.assertTrue(result["ok"], result)
        client.login(name, "demo123456")
        inbox.wait("result", lambda b: b["action"] == "LOGIN")
        return client, inbox

    def socket(self):
        sock = socket.create_connection(("127.0.0.1", self.port), timeout=3)
        self.raw.append(sock)
        return sock

    def response(self, sock, task, body=None):
        send_frame(sock, make_packet(task, body))
        return recv_frame(sock)

    def test_04_register_login_password_and_persistence(self):
        client, box = self.account("账户校验")
        stored = json.loads(self.db.read_text(encoding="utf-8"))["账户校验"]
        self.assertNotIn("demo123456", self.db.read_text(encoding="utf-8"))
        expected = hashlib.pbkdf2_hmac("sha256", b"demo123456", bytes.fromhex(stored["salt"]), 2000
 > 00).hex()
        self.assertEqual(stored["digest"], expected)
        sock = self.socket()
        self.assertFalse(self.response(sock, "REGISTER", dict(name="账户校验", password="demo123456
 > "))["body"]["ok"])
        self.assertEqual(self.response(sock, "LOGIN", dict(name="账户校验", password="wrong123"))["
```

```
 >  body"]["code"], "AUTH_FAILED")
124         self.assertEqual(self.response(sock, "LOGIN", dict(name="账户校验", password="demo123456"))
 >  ["body"]["code"], "ALREADY_ONLINE")
125         client.logout()
126         box.wait("result", lambda b: b["action"] == "LOGOUT")
127         self.assertTrue(self.response(sock, "LOGIN", dict(name="账户校验", password="demo123456"))[
 >  "body"]["ok"])
128 
129     def test_05_real_multicast_chinese_and_dedup(self):
130         a, ia = self.account("组播甲")
131         b, ib = self.account("组播乙")
132         text = "中文群聊：你好，世界！English 123 🙂"
133         a.chat(text)
134         ma = ia.wait("message", lambda m: m["text"] == text)
135         mb = ib.wait("message", lambda m: m["text"] == text)
136         self.assertEqual(ma["transport"], "UDP 组播")
137         self.assertEqual(mb["transport"], "UDP 组播")
138         self.assertEqual(ma["id"], mb["id"])
139         # A repeated server packet must not display twice.
140         repeated = make_packet("MESSAGE", {k: v for k, v in ma.items() if k not in {"id", "ts", "tr
 >  ansport"}})
141         repeated["id"] = ma["id"]
142         a._message(repeated, "TCP")
143         self.assertFalse(ia.has("message", lambda m: m["text"] == text))
144 
145     def test_06_selected_multiple_recipient_isolation(self):
146         a, ia = self.account("定向甲")
147         b, ib = self.account("定向乙")
148         c, ic = self.account("定向丙")
149         d, id_ = self.account("旁观丁")
150         a.chat("仅乙丙收到", ["定向乙", "定向丙"])
151         for box in (ia, ib, ic):
152             message = box.wait("message", lambda m: m["text"] == "仅乙丙收到")
153             self.assertEqual(message["transport"], "TCP")
154             self.assertEqual(message["mode"], "selected")
155         with self.assertRaises(AssertionError):
156             id_.wait("message", lambda m: m["text"] == "仅乙丙收到", timeout=0.35)
157 
158     def test_07_multicast_loss_tcp_repair(self):
159         a, ia = self.account("补发甲")
160         b, ib = self.account("补发乙", multicast=False)
161         start = time.monotonic()
162         a.chat("关闭组播仍收到")
163         msg = ib.wait("message", lambda m: m["text"] == "关闭组播仍收到")
164         self.assertEqual(msg["transport"], "TCP")
165         self.assertGreaterEqual(time.monotonic() - start, 0.65)
166 
167     def test_08_invalid_task_and_corruption_recover(self):
168         sock = self.socket()
169         self.assertEqual(self.response(sock, "TASK_DOES_NOT_EXIST")["body"]["code"], "INVALID_TASK"
 >  )
170         bad = make_packet("PING")
171         bad["body"]["tampered"] = True
172         raw = canonical(bad)
173         sock.sendall(struct.pack("!I", len(raw)) + raw)
174         self.assertEqual(recv_frame(sock)["body"]["code"], "BAD_CHECKSUM")
175         self.assertEqual(self.response(sock, "PING")["type"], "PONG")
176 
177     def test_09_fragmented_and_coalesced_tcp_frames(self):
178         sock = self.socket()
179         one, two = encode(make_packet("PING")), encode(make_packet("PING"))
180         frame = struct.pack("!I", len(one)) + one + struct.pack("!I", len(two)) + two
181         for index in range(0, len(frame), 7):
182             sock.sendall(frame[index:index+7])
183         self.assertEqual(recv_frame(sock)["type"], "PONG")
184         self.assertEqual(recv_frame(sock)["type"], "PONG")
```

```
185
186     def test_10_malformed_packet_and_oversized_frame(self):
187         sock = self.socket()
188         for raw in (b"not JSON", b"\xff", b'{"v":1,"v":2}'):
189             sock.sendall(struct.pack("!I", len(raw)) + raw)
190             self.assertEqual(recv_frame(sock)["body"]["code"], "BAD_PACKET")
191         self.assertEqual(self.response(sock, "CHAT", dict(text="匿名", mode="all"))["body"]["code"]
  > , "NOT_LOGGED_IN")
192         sock.sendall(struct.pack("!I", MAX_FRAME + 1))
193         self.assertEqual(recv_frame(sock)["body"]["code"], "FRAME_TOO_LARGE")
194         self.assertEqual(sock.recv(1), b"")
195         fresh = self.socket()
196         self.assertEqual(self.response(fresh, "PING")["type"], "PONG")
197
198     def test_11_bad_body_replay_and_roster_on_disconnect(self):
199         a, ia = self.account("边界甲")
200         b, ib = self.account("边界乙")
201         ia.wait("roster", lambda m: "边界乙" in m["members"])
202         for text, targets, code in [("", None, "BAD_BODY"), ("字" * 2001, None, "BAD_BODY"),
203                                     ("你好", [], "NO_RECIPIENT"), ("你好", ["不存在"], "NO_RECIPIEN
  > T")]:
204             a.chat(text, targets)
205             ia.wait("error", lambda m: m["code"] == code)
206         a.request("CHAT", dict(text="类型错误", mode="selected", targets=[123]))
207         ia.wait("error", lambda m: m["code"] == "BAD_BODY")
208         b.close()
209         ia.wait("roster", lambda m: "边界乙" not in m["members"])
210         raw = self.socket()
211         packet = make_packet("PING")
212         send_frame(raw, packet)
213         self.assertEqual(recv_frame(raw)["type"], "PONG")
214         send_frame(raw, packet)
215         self.assertEqual(recv_frame(raw)["body"]["code"], "REPLAY")
216
217     def test_12_database_survives_restart(self):
218         # Separate real process proves persistence independent of server memory.
219         temp = tempfile.TemporaryDirectory(prefix="lan_persist_")
220         with socket.socket() as probe:
221             probe.bind(("127.0.0.1", 0))
222             port = probe.getsockname()[1]
223         server = NativeServer("127.0.0.1", port, Path(temp.name) / "accounts.json", "127.0.0.1", la
  > mbda x: None)
224         try:
225             server.start()
226             with socket.create_connection(("127.0.0.1", port), timeout=3) as s:
227                 self.assertTrue(self.response(s, "REGISTER", dict(name="持久化", password="persist1
  > 23"))["body"]["ok"])
228             server.stop()
229             server.start()
230             with socket.create_connection(("127.0.0.1", port), timeout=3) as s:
231                 self.assertTrue(self.response(s, "LOGIN", dict(name="持久化", password="persist123"
  > ))["body"]["ok"])
232         finally:
233             server.stop()
234             temp.cleanup()
235
236
237 if __name__ == "__main__":
238     unittest.main(verbosity=2)
```